Failure Synthesis System

Python Implementation Handoff

1. Role of Python

Java owns the **event system**:

- event ingestion
- binary event validation/framing
- crash-safe storage
- time-based segments
- per-agent blocks
- block metadata
- bounded retrieval

Python owns the **intelligence layer**:

Java Retrieval
↓
EventRecord
↓
Deterministic Decoder
↓
Structured Events
↓
Domain Knowledge / RAG
↓
LLM
↓
Failure Synthesis

The important boundary is:

> **Java determines what evidence exists and retrieves it correctly. Python determines how that evidence is interpreted and synthesized.**

Python should **not directly access Java's storage files or internal block/segment structures.**

---

2. Java → Python Retrieval Contract

The retrieval interface is:

```
retrieve(
   source_id | all,
   start_time,
   end_time,
   event_type?
)
```

Examples:

```
retrieve(
   source_id=474,
   start_time=T1,
   end_time=T2
)
```

or:

```
retrieve(
   source_id=all,
   start_time=T1,
   end_time=T2
)
```

or:

```
retrieve(
   source_id=all,
   start_time=T1,
   end_time=T2,
   event_type="POSITION"
)
```

Important constraint

Every retrieval request **must contain a bounded time range**.

There is no:

```
retrieve(all_history)
```

or an unbounded request.

This is a hard safety/performance boundary so that an intelligence/LLM layer cannot accidentally request the entire event history.

---

3. What Java Does Internally

Python does not need to know this, but the retrieval flow is:

time range
↓
relevant segments
↓
source → block IDs
↓
block metadata
↓
relevant blocks
↓
event scan
↓
matching EventRecords

Each block contains events belonging to one source/agent.

Blocks contain only complete events.

If an event does not fit in the remaining capacity of a block:

current block
↓
remaining space too small
↓
leave unused capacity
↓
write entire event into next block

Therefore Python can assume that every returned event is a complete event.

---

4. EventRecord

Java returns **events**, not storage structures.

Conceptually:

```
class EventRecord:
    source_id
    timestamp
    event_type
    payload_bytes
```

Equivalent representation could simply be a dataclass:

```
@dataclass
class EventRecord:
    source_id: int
    timestamp: int
    event_type: int | str
    payload_bytes: bytes
```

The exact timestamp representation can be chosen consistently between Java and Python, but it must preserve the original **producer timestamp**.

Fields

`source_id`

Identifies the producer/agent that generated the event.

Example:

```
474
298
901
```

This is important because failure explanation may involve multiple agents.

`timestamp`

The timestamp attached by the producer.

It is **not automatically replaced by Java's receive time**.

Python should preserve it as evidence.

We do not assume:

- perfectly synchronized clocks across drones

- globally monotonic timestamps
- exact causal ordering across agents

Therefore retrieval does not need to promise global timestamp ordering.

`event_type`

Identifies the event/message type.

Example:

POSITION
BATTERY_STATUS
LIDAR_FAULT
PLANNER_STATE
MOTOR_STATUS

Actual IDs/names depend on the deployment protocol.

The important purpose is:

event_type
↓
select deterministic decoder/schema

`payload_bytes`

The original binary payload.

Example:

payload_bytes: bytes

Java should not convert this into semantic Python objects.

Python is responsible for decoding it.

---

5. Java's Guarantee

The core contract is:

> **Java guarantees that every returned EventRecord corresponds to a valid stored event satisfying the retrieval constraints. Python is responsible for interpreting its payload.**

Java does **not** guarantee that:

- events are globally timestamp-sorted
- all event types are known to Python
- every payload can be decoded
- every byte has known semantic meaning

---

6. Python Deterministic Decoder

The first Python component after retrieval should be a **deterministic binary decoder**.

EventRecord
↓
event_type
↓
schema lookup
↓
binary decoder
↓
DecodedEvent

The LLM must **not** perform binary decoding.

Do not ask an LLM to figure out:

bytes 0-3 = uint32?
bytes 4-7 = float?
little endian?
big endian?
bit 3 means what?

That must be determined by code/configuration.

---

7. Schema Definition

Each supported event type has a schema describing how its payload is decoded.

Conceptually:

EventSchema:
  event_type
  fields

Each field needs enough information for deterministic decoding:

FieldDefinition:
  name
  offset
  type
  size
  endian

For example:

LIDAR_STATUS

field: status_code
offset: 0
type: uint16
size: 2
endian: little

field: timeout_count
offset: 2
type: uint32
size: 4
endian: little

The exact schema format can be JSON/YAML/configuration or Python structures.

Don't build a giant schema framework.

The goal is simply to give the decoder exact instructions for supported event types.

---

8. Primitive Types

The decoder should support the primitive types actually needed by the event schemas.

For example:

uint8
uint16
uint32
uint64

int8
int16
int32
int64

float32
float64

boolean

Additional types can be added only when a real event schema requires them.

The decoder must explicitly know:

- byte offset
- byte size
- signed/unsigned
- integer/float
- endianness

No guessing.

---

9. Enums / Error Codes

Some fields may represent enumerated values.

Example:

status_code = 17

A domain/schema definition might specify:

17 → LIDAR_TIMEOUT
18 → SENSOR_DISCONNECTED
19 → INVALID_SCAN

The deterministic decoder can produce:

```
{
   "status_code": 17,
   "status_name": "LIDAR_TIMEOUT"
}
```

if the mapping is configured.

The LLM should not invent:

17 probably means lidar timeout

If the mapping is unknown:

status_code = 17
status_name = UNKNOWN

and it remains unknown.

---

10. Unknown Event Types

If Java returns:

event_type = 0x31

but Python has no schema for `0x31`:

Do not guess.

Return something like:

```
DecodedEvent(
   source_id=474,
   timestamp=...,
   event_type=0x31,
   fields={},
   decode_status="UNKNOWN_EVENT_TYPE"
)
```

The raw payload can remain preserved for traceability, but it should not be presented to the LLM as something the LLM is expected to decode.

---

11. Unknown Payload Bytes

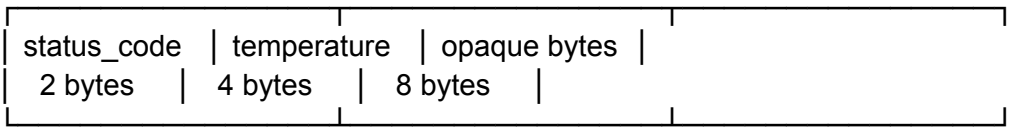
A payload may contain:

[known fields][unknown/opaque bytes]

The deterministic decoder should decode the known portion.

Example:

payload:



Result:

```
{
   "status_code": 17,
   "temperature": 72.4
}
```

with:

decode_status = PARTIALLY_DECODED
unknown_bytes = 8

The opaque bytes are **not interpreted by the LLM**.

Dashboard can report:

8 bytes of event payload could not be interpreted because
their structure is not configured/documented.

---

12. Malformed Payload Handling

Decoder must validate bounds before reading.

For example, if schema says:

offset = 8
size = 4

but payload has only 10 bytes:

required: bytes 8–11
available: bytes 8–9

then decoding fails deterministically.

Do not partially invent the missing value.

Return something like:

decode_status = MALFORMED_PAYLOAD

and record the reason.

The LLM should be told that the field could not be decoded rather than being given a fabricated value.

---

13. DecodedEvent

After deterministic decoding, Python should convert the event into a structured representation.

Conceptually:

```
class DecodedEvent:
    source_id
    timestamp
    event_type
    fields
    decode_status
    warnings
```

Example:

```
{
    "source_id": 474,
```

```
  "timestamp": 1723456789123,
  "event_type": "LIDAR_STATUS",
  "fields": {
    "status_code": 17,
    "status_name": "LIDAR_TIMEOUT",
    "timeout_count": 3
  },
  "decode_status": "SUCCESS",
  "warnings": []
}
```


This structured representation is what should be passed onward to the intelligence layer.

---

14. Data Confidence Levels

The system should distinguish between three kinds of information:

1. Deterministically decoded

Example:

battery_voltage = 21.4 V

This came directly from deterministic binary decoding.

Treat it as factual event evidence.

2. Documented semantic interpretation

Example:

status_code 17 = LIDAR_TIMEOUT

This comes from configured/domain documentation.

Treat it as documented domain meaning.

3. Unknown

Example:

payload contains 8 unknown bytes

The system explicitly does not know what those bytes mean.

Do not let the LLM turn this into a confident interpretation.

---

15. Domain Knowledge / RAG

After decoding:

Decoded Events
+
Domain Knowledge
↓
RAG context
↓
LLM

Domain knowledge can contain things such as:

error-code definitions
protocol documentation
vehicle/robot manuals
planner-state descriptions
system architecture documentation
failure-mode documentation

The distinction is:

Schema
→ tells Python HOW to decode bytes

Domain Knowledge
→ tells the system WHAT decoded values mean

For example:

bytes
↓
uint16 = 17
↓
schema says field = status_code
↓
domain documentation says
17 = LIDAR_TIMEOUT

↓
LLM can reason about LIDAR_TIMEOUT

---

16. LLM Responsibility

The LLM is responsible for **reasoning and synthesis**, not correctness-critical parsing.

It can answer questions such as:

What likely caused Drone 474 to leave formation?

Which events support this explanation?

Was the failure preceded by a sensor anomaly?

Did another drone contribute to the incident?

What evidence is missing?

It should reason only over the structured evidence and retrieved domain knowledge supplied to it.

---

17. Retrieval Is Still Controlled Outside the LLM

The LLM may request additional context, but it does not get direct storage access.

Example:

Initial query
↓
Java retrieval
↓
small evidence set
↓
Python decoding
↓
LLM
↓

"I need context before this event"
↓
bounded retrieval request
↓
Java

Any request must still satisfy:

source_id | all
start_time
end_time
event_type?

and the retrieval layer enforces result/context limits.

Possible progression:

initial window
↓
narrow investigation
↓
wider bounded window if required
↓
final synthesis

This is optional intelligence behavior, not a correctness requirement.

---

18. Cross-Agent Investigation

Do not assume that the queried agent is necessarily the cause.

Example:

Drone 474
↓
unexpected avoidance maneuver

The cause could be:

Drone 298
↓
LiDAR malfunction

↓
trajectory deviation
↓
Drone 474 detects conflict
↓
avoidance maneuver

Therefore retrieval supports:

source_id = all

within a bounded time range.

Python can then provide the resulting multi-agent evidence to the reasoning layer.

No hardcoded assumption should be made that a particular event type or drone relationship exists.

---

19. Important Clock Assumption

Producer timestamps are preserved, but the system does **not** implement clock synchronization.

Do not assume:

Drone 298 timestamp 10:07:03.050
<
Drone 474 timestamp 10:07:03.120

necessarily means Drone 298's event caused Drone 474's event.

The timestamps are evidence, not a guaranteed globally synchronized causal timeline.

If necessary, the final synthesis should state uncertainty.

---

20. LLM Context Format

The exact prompt/template can evolve, but conceptually the LLM receives:

USER QUERY

+

DECODED EVENTS

+

DOMAIN KNOWLEDGE

+

DATA QUALITY / DECODING WARNINGS

Example:

User:
Why did Drone 474 leave formation?

Events:

[10:07:03.050]
source=298
event=LIDAR_STATUS
status=LIDAR_TIMEOUT

[10:07:03.120]
source=474
event=AVOIDANCE
reason=COLLISION_RISK

[10:07:03.180]
source=474
event=FORMATION_STATE
state=LEFT_FORMATION

Domain knowledge:

LIDAR_TIMEOUT:
A LiDAR timeout indicates that the expected scan was not
received within the configured timeout interval.

Evidence limitations:

8 bytes of one diagnostic event were not decoded because
their structure is unknown.

The LLM then synthesizes the explanation.

---

21. Output Contract

The final synthesis should not just be a paragraph.

Conceptually:

```
FailureSynthesis:
   summary
   contributing_events
   likely_cause
   confidence
   uncertainty
   data_gaps
```

Example:

```
{
   "summary": "Drone 474 left formation after detecting a collision risk.",
   "likely_cause": "A LiDAR timeout on Drone 298 appears to have preceded a trajectory
deviation that triggered avoidance behavior on Drone 474.",
   "confidence": "medium",
   "contributing_events": [
      {
         "source_id": 298,
         "timestamp": "...",
         "event_type": "LIDAR_STATUS"
      },
      {
         "source_id": 474,
         "timestamp": "...",
         "event_type": "AVOIDANCE"
      }
   ],
   "uncertainty": [
      "Producer clocks are not guaranteed to be globally synchronized."
   ],
   "data_gaps": [
      "One diagnostic payload contained 8 uninterpreted bytes."
   ]
}
```

The exact JSON/schema can be finalized when the dashboard implementation starts.

---

22. Hard Rules

These are the important things that should **not** be violated during implementation.

1. Python never reads Java storage files directly.

2. Java owns retrieval.

3. Every retrieval request has a bounded time range.

4. Java returns complete EventRecords, not storage blocks.

5. Producer timestamp is preserved.

6. Java does not promise global timestamp ordering.

7. Payload remains binary until Python decoding.

8. Binary decoding is deterministic code/configuration.

9. LLM never decides byte offsets, types, endianness, etc.

10. Unknown event types are not guessed.

11. Unknown payload regions remain opaque.

12. Unknown bytes are not given to the LLM for interpretation.

13. Domain documentation supplies semantic meaning.

14. LLM reasons over decoded evidence; it does not establish raw-data meaning.

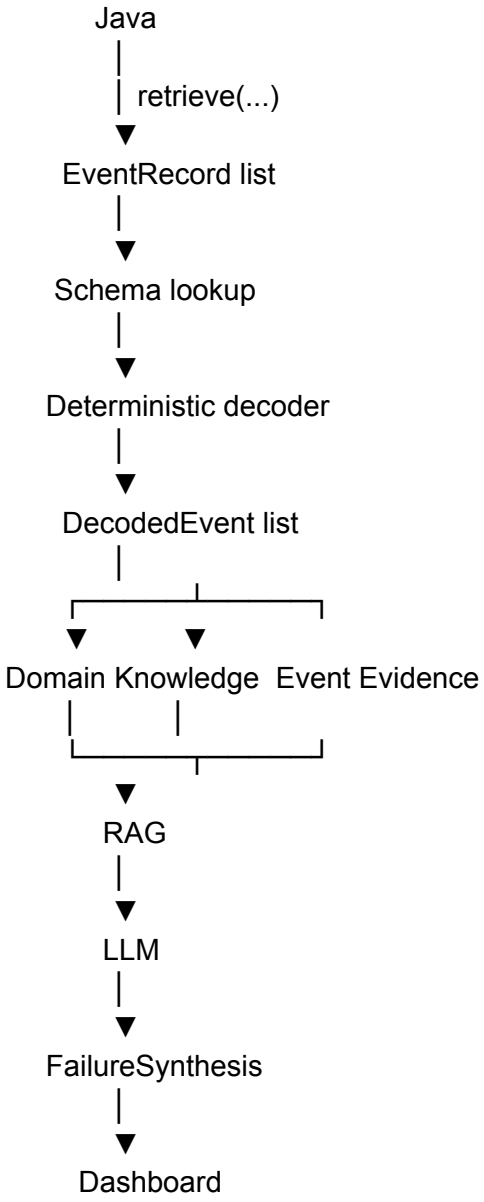
15. Cross-agent retrieval is allowed through source_id=all.

16. Storage/retrieval internals remain hidden from Python.

---

23. Minimal Python Pipeline

The implementation should initially be kept this simple:



First implementation milestone

Do **not** start with the LLM.

First make this work:

mock EventRecord
↓
schema lookup
↓
deterministic binary decode
↓
DecodedEvent

Then:

DecodedEvent
↓
domain knowledge retrieval
↓
LLM

Then connect the real Java retrieval interface.

That keeps the Python side independently testable and makes the Java/Python boundary very clean.